

Day $\Rightarrow$ 1

Collection

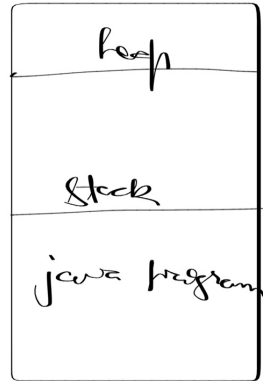
Framework $\Rightarrow$ A framework is a pre-written, reusable software platform. It provides a foundation / structure for developing applications effectively.

Collection in Java $\Rightarrow$

(Data structure)

Arrays

- $\rightarrow$ fixed size
 - `int [ ] arr = new int [3];`
// if it store 5 so
create specific array
- $\rightarrow$ homogeneous data storage
- $\rightarrow$ not fast searching



so array has a lot of limitations so we require more data structure for

- $\rightarrow$ no prior knowledge of data size
- $\rightarrow$ automatically extend / shrink
- $\rightarrow$ fast searching
- $\rightarrow$ Ordered / unordered
- $\rightarrow$ Duplicate / unique
- $\rightarrow$ key value pair storage

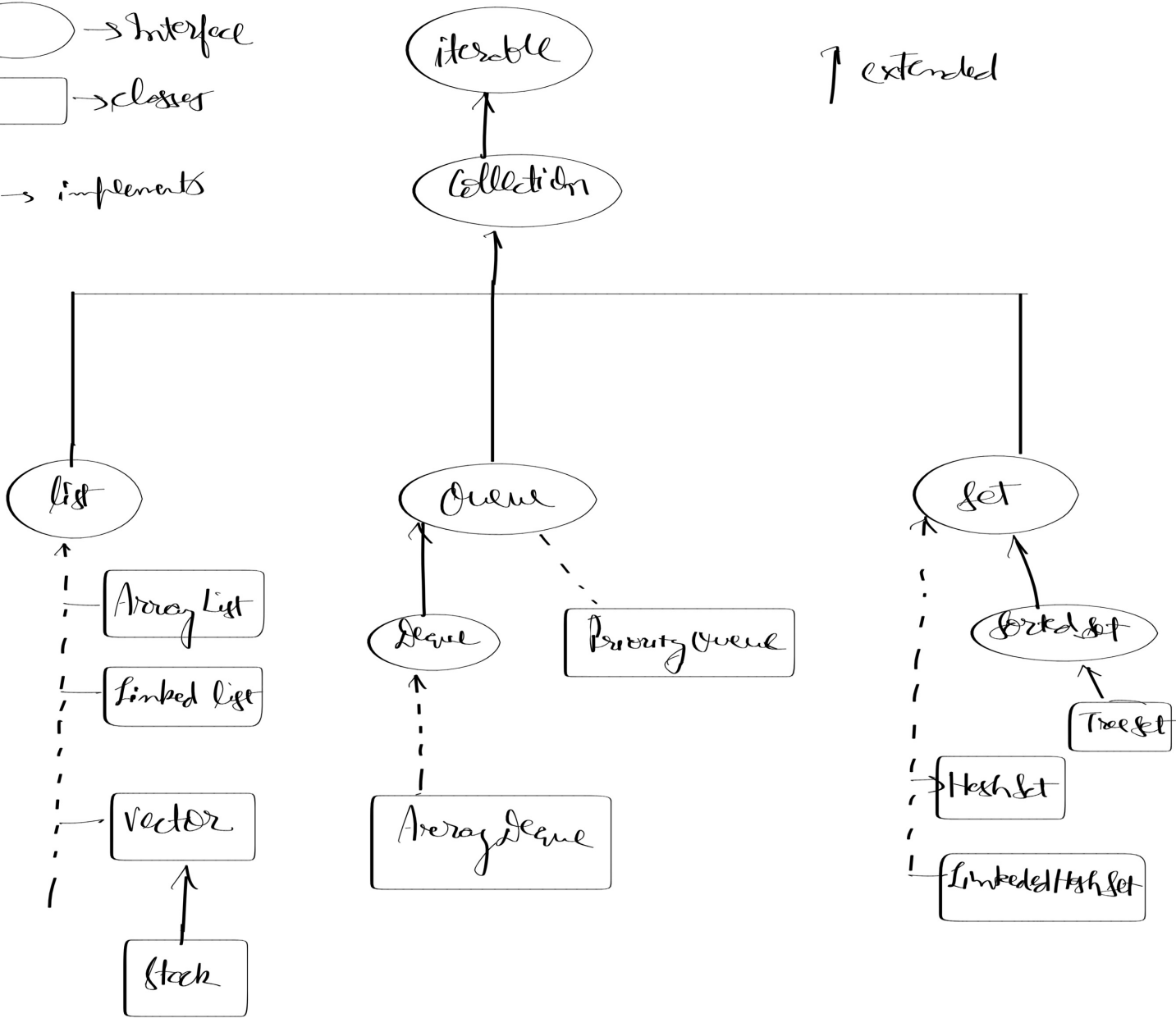
so we need Java collection framework

○ → Interface

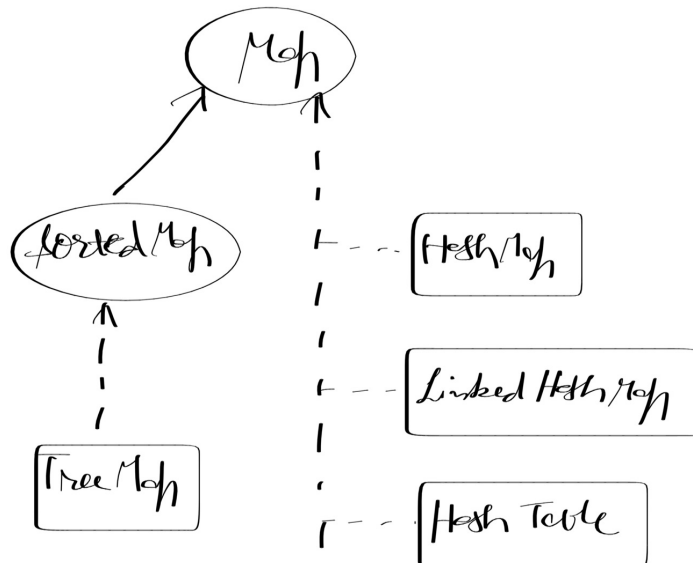
□ → classes

↑ → implements
- - -

↑ extended



Note → Map is also part of Java Collection framework



Java Collection Interface

java.util

Collection



Interface

(An interface is a blueprint of a class. It defines a set of methods without implementation)

The collection interface is the foundation of the Java Collection framework. It represents a group of objects as a single unit.

Collection



class

Methods of Collection

(1) $\text{add}(E\ e)$ ^{→ Type} _{→ element} → return type true/false

Suppose we have class ArrayList

C1 →

10	5	7	8	15	10	
----	---	---	---	----	----	--

C2 →

7	9	11
---	---	----

C1.add(10)

Syntax =

```
ArrayList<String> list = new ArrayList<>();  
list.add("16");
```

② `addAll(Collection c)` → return true/false

```
ArrayList<String> list2 = new ArrayList<>();  
list2.addAll(list);
```

→ Note ⇒ It will work only if the array list of same generic

Actual syntax ⇒

`addAll(Collection<? extends E> c)`

```
1 import java.util.ArrayList;  
2 class Main{  
3     public static void main(String[] args) {  
4         ArrayList<String>list=new ArrayList<>();  
5         list.add("bhupesh");  
6         System.out.println(list.add("bhupesh"));  
7         ArrayList<String>list2=new ArrayList<>();  
8         list2.add("2");  
9         // if we want to add all elements  
10  
11         list2.addAll(list);  
12         System.out.println(list2.addAll(list));  
13  
14         //print arraylist  
15         System.out.println(list);  
16         System.out.println(list2);  
17  
18     }  
19 }
```

Handwritten annotations on the screenshot:

- Next to line 5: `list.add("bhupesh");` → `list` → `[bhupesh | bhupesh]`
- Next to line 8: `list2.add("2");` → `list2` → `[2]`
- Next to line 11: `list2.addAll(list);` → `[2 | bhupesh | bhupesh]`
- Next to line 16: `System.out.println(list2);` → `[2 | bhupesh | bhupesh | bhupesh | bhupesh]`

Remove elements from list

list2.remove("bhupesh");

System.out.println(list2.remove("bhupesh"));

```
Main.java x
2  class Main{
3      public static void main(String[] args) {
17
18          // removing the element
19          list2.remove(o: "bhupesh");
20          System.out.println(list2.remove(o: "bhupesh"));
21          // if elements is not present in it then it will return false
22          System.out.println(list2.remove(o: "bkk"));
23
24          // removing all elemnts from the list
25          System.out.println(list.removeAll(list));
26          System.out.println(list);
27      }
28  }
```

```
true
true
[bhupesh, bhupesh]
[2, bhupesh, bhupesh, bhupesh, bhupesh]
true
false
true
[]

Process finished with exit code 0
```

To see is there any elements in list or not

`collection_name.isEmpty();`

to return size of the list

`collection_name.size();`

Note =>

Collection<Integer> l1 = new ArrayList<>();



Is it right? Yes

Collection<Integer> (l1) → reference variable of class which is above

new ArrayList<>(); → object is created of below class

```
38
39      Collection<Integer> l1=new ArrayList<>();
40      l1.add(1);
41      l1.add(2);
42      System.out.println(l1);
43  }
44 }
```

To clear all elements

collection_name.clear();

```
44      // to make all element delete we also use clear
45      l1.clear();
46      System.out.println(l1);
47  }
48 }
```

To see whether list contains elements or not → true / false

collection_name.contains(object);

```
// to see whether the list contains an element -> .contains(object)
System.out.println(l1.contains(1));
}
```

⇒ Contains All — $\begin{cases} \rightarrow \text{time} \\ \rightarrow \text{space} \end{cases}$

C_1 contains $All(C_2)$ $\rightarrow$ whether C_1 contains all elements of C_2

⇒ boolean equals (Object)

Collection Equality

Implementation

Each collection type defines specific equality semantics

Return

returns true if the specified object is equal to this collection

```
48 // another method equals(object)  
49 List<String> a1=new ArrayList<>();  
50 a1.add("bhupesh");  
51 a1.add("kumar");  
52 a1.add("karir");  
53 Set<String>s1=new HashSet<>();  
54 s1.add("karir");  
55 s1.add("kumar");  
56 s1.add("bhupesh");  
57  
58  
59 System.out.println(a1.equals(s1));  
60 System.out.println(s1.equals(a1));  
61  
62 }  
63
```

false

false

```
Process finished with exit c
```

Java Collections — ArrayList, List & Set

Exam Notes with Examples and Outputs

1. Important ArrayList Methods

Method	Purpose	Return Type
add()	Adds one element to the list	boolean
addAll()	Adds all elements of another collection	boolean
remove()	Removes one matching element	boolean / element depending on overload
removeAll()	Removes all elements that match a collection	boolean
isEmpty()	Checks whether the list has no elements	boolean
size()	Returns the number of elements	int

2. add() and Duplicate Elements

ArrayList allows duplicate elements. add() normally returns true when the element is successfully added.

```
ArrayList<String> list = new ArrayList<>();
list.add("bhupesh");
System.out.println(list.add("bhupesh"));
```

Output:
true

```
list = [bhupesh, bhupesh]
```

3. addAll()

addAll() adds every element of another collection. It does not replace the existing elements.

```
list = [bhupesh, bhupesh]
list2 = [2]
```

```
list2.addAll(list);
```

```
list2 = [2, bhupesh, bhupesh]
```

If addAll(list) is called again, the same elements are added again.

4. remove()

When removing by object, remove("bhupesh") removes only the first matching occurrence. If the element is absent, it returns false.

```
list2 = [2, bhupesh, bhupesh, bhupesh]
```

```
list2.remove("bhupesh");
list2 = [2, bhupesh, bhupesh]
```

```
System.out.println(list2.remove("bkk"));
```

Output:
false

5. removeAll()

removeAll(collection) removes all elements from the current collection that are also present in the supplied collection.

```
list = [bhupesh, bhupesh]

System.out.println(list.removeAll(list));
Output:
true

System.out.println(list);
Output:
[]
```

6. isEmpty() and size()

```
list.isEmpty()
→ true if the list contains 0 elements

list.size()
→ returns the number of elements

Example:
list = []
list.isEmpty() → true
list.size()    → 0
```

7. List vs Set — equals()

A List is ordered and allows duplicates. A Set does not use list-style positional equality and does not allow duplicate elements.

```
List<String> a1 = new ArrayList<>();
a1.add("bhupesh");
a1.add("kumar");
a1.add("karir");

Set<String> s1 = new HashSet<>();
s1.add("karir");
s1.add("kumar");
s1.add("bhupesh");

System.out.println(a1.equals(s1));
System.out.println(s1.equals(a1));

Output:
false
false
```

Why false?

Even though both collections contain the same three strings, a List and a Set are different collection types and their equals() methods use different equality rules. Therefore List.equals(Set) and Set.equals(List) are false.

8. List Equality — Order Matters

```
List: [1, 2, 3]
List: [1, 2, 3]
→ equal

List: [1, 2, 3]
List: [3, 2, 1]
```

→ not equal

For Lists:

Same elements + same order → equal

9. Set Equality — Order Does Not Matter

Set: {1, 2, 3}

Set: {3, 2, 1}

→ equal

For Sets:

Same elements → equal, regardless of iteration/order.

Quick Exam Rules

1. ArrayList preserves insertion order.
2. ArrayList allows duplicate elements.
3. `addAll()` adds all elements; it does not replace existing elements.
4. `remove(Object)` removes the first matching occurrence.
5. `removeAll()` can remove multiple matching elements.
6. `isEmpty()` returns true when size is 0.
7. `size()` returns the number of elements.
8. List equality is order-sensitive.
9. Set equality is based on elements, not positional order.
10. A List and a Set containing the same elements are not equal to each other.



$c_1.removeAll(c_2);$



remove c_2 from c_1

c_1 [a|b|c|d|e]

c_2 [b|c|d|e]

c_1 [a]

$c_1.retainAll(c_2);$



elements retain
which in c_2 in c_1
and rest all deleted

c_1 [a|b|c|d|e]

c_2 [c|d|e|f]

$c_1 \rightarrow$ [c|d|e]

```
// remove all  
a1.removeAll(s1);  
System.out.println(a1);
```

```
// retain all  
s1.retainAll(a1);  
System.out.println(s1);
```

}

}



[]

[]



```
// concept of iterator
List<String> b1=new ArrayList<>();
b1.add("1");
b1.add("2");
b1.add("bhupesh");
// as iterator is an interface
Iterator<String>itr= b1.iterator();
while(itr.hasNext())
    System.out.println(itr.next());
```



}

}



1

2

bhupesh

Process finished with exit code 0